

```

default_nettype none
`include "core.v"
`include "Instr_Memory.v"
`include "data_memory.v"

// Include the parameters file for instruction definitions
`include "parameters.vh"

module top_module(
    input wire clk,
    input wire rst_n,
    output wire [31:0] ex_result,
    output wire [31:0] if_instr,
    output wire [31:0] if_Pc,
    output wire [31:0] id_rs1_data,
    output wire [31:0] id_rs2_data,
    output wire [31:0] id_imm_data,
    output wire [31:0] id_pc,
    output wire [2:0] id_func3,
    output wire id_regSrc,
    output wire id_sel1,
    output wire id_sel2,
    output wire id_load,
    output wire id_branch,
    output wire id_jal,
    output wire id_jalr,
    output wire id_memSrc,
    output wire [1:0] id_resultSrc,
    output wire [3:0] id_alu_ctrl,
    output wire [31:0] ex_pc,
    output wire [31:0] ex_imm_data,
    output wire [31:0] ex_data_store,
    output wire [2:0] ex_func3,
    output wire ex_load,
    output wire [31:0] result_to_if,
    output wire ex_boj,
    output wire ex_jalr,
    output wire ex_regsrc,
    output wire ex_memsrc,
    output wire [1:0] ex_resultsrc,
    output wire [4:0] m_rd,
    output wire m_regSrc,
    output wire [1:0] m_resultsrc,
    output wire [31:0] m_result,
    output wire [31:0] m_data,
    output wire [31:0] m_pc4,

```

```

output wire w_regsrsrc,
output wire [31:0] w_data,
output wire [4:0] id_rd,
output wire [4:0] ex_rd,
output wire [4:0] w_rd,
output wire [31:0] fwdwire2, fwdwire1,
output wire o_load_use_hazard
);

wire [31:0] i_i_data;
wire [31:0] i_d_data;
wire i_i_ack;
wire i_d_ack;
wire o_i_stb;
wire o_d_stb;
wire o_d_wr_en;
wire [31:0] o_d_write_data;
wire [31:0] o_i_addr;
wire [31:0] o_d_addr;

// Instantiate the core, passing the instructions and opcodes
core core_inst (
    .clk(clk),
    .rst_n(rst_n),
    .i_i_data(i_i_data),
    .i_d_data(i_d_data),
    .i_i_ack(i_i_ack),
    .i_d_ack(i_d_ack),
    .o_i_stb(o_i_stb),
    .o_d_stb(o_d_stb),
    .o_d_wr_en(o_d_wr_en),
    .o_d_write_data(o_d_write_data),
    .o_i_addr(o_i_addr),
    .o_d_addr(o_d_addr),
    .ex_result(ex_result),

    // IF Stage Outputs
    .if_instr(if_instr),
    .if_Pc(if_Pc),

    // ID Stage Outputs
    .id_rs1_data(id_rs1_data),
    .id_rs2_data(id_rs2_data),
    .id_imm_data(id_imm_data),
    .id_pc(id_pc),
    .id_func3(id_func3),

```

```

.id_rd(id_rd),
.id_regSrc(id_regSrc),
.id_sel1(id_sel1),
.id_sel2(id_sel2),
.id_load(id_load),
.id_branch(id_branch),
.id_jal(id_jal),
.id_jalr(id_jalr),
.id_memSrc(id_memSrc),
.id_resultSrc(id_resultSrc),
.id_alu_ctrl(id_alu_ctrl),

// EX Stage Outputs
.ex_pc(ex_pc),
.ex_imm_data(ex_imm_data),
.ex_data_store(ex_data_store),
.ex_func3(ex_func3),
.ex_rd(ex_rd),
.ex_load(ex_load),
.result_to_if(result_to_if),
.ex_boj(ex_boj),
.ex_jalr(ex_jalr),
.ex_regsrc(ex_regsrc),
.ex_memsrc(ex_memsrc),
.ex_resultsrc(ex_resultsrc),

// MEM Stage Outputs
.m_regSrc(m_regSrc),
.m_rd(m_rd),
.m_resultsrc(m_resultsrc),
.m_result(m_result),
.m_data(m_data),
.m_pc4(m_pc4),

// WB Stage Outputs
.w_rd(w_rd),
.w_regsrc(w_regsrc),
.w_data(w_data),

// Forwarding and Hazard Signals
.fwdwire2(fwdwire2),
.fwdwire1(fwdwire1),
.o_load_use_hazard(o_load_use_hazard)
);

```

```

);

// Data Memory Module

```

```
data_mem data_mem_inst (  
    .clk(clk),  
    .rst_n(rst_n),  
    .i_stb(o_d_stb),  
    .i_wr_en(o_d_wr_en),  
    .i_addr(o_d_addr),  
    .i_write_data(o_d_write_data),  
    .o_rd_ack(i_d_ack),  
    .o_read_data(i_d_data)  
);
```

```
// Instruction Memory Module
```

```
instr_mem instr_mem_inst (  
    .rst_n(rst_n),  
    .i_addr(o_i_addr),  
    .i_stb(o_i_stb),  
    .o_ack(i_i_ack),  
    .o_data(i_i_data)  
);
```

```
endmodule
```